

Diagnosing Tool-Call Decision Making in Small Language Models

Anirudh Poruri
aporuri@umd.edu

Mukund Shankar
mukunds@umd.edu

Abstract

Tool-calling language models must decide whether to call a tool, ask for missing information, or state that the available tools are insufficient. We study this three-way decision problem in small language models using Gemma-3-4B-it and Llama-3.2-3B-Instruct. We compare prompting, linear probes over hidden states, supervised fine-tuning (SFT), Direct Preference Optimization (DPO), and a Constitutional AI (CAI)-style pipeline based on rule-guided self-critique, self-revision, and preference optimization. On When2Call, standard SFT yields the strongest Llama performance, while CAI-DPO on the base model yields the strongest performance for Gemma. Linear probes often recover the correct decision more reliably than the models’ final outputs do, suggesting that some failures occur even when decision-relevant information is present in the hidden states. Overall, labeled SFT is the most reliable path to balanced behavior when labeled data is available; CAI-style self-supervision is not a drop-in replacement, as it can turn missing information and insufficient tool examples into plausible but incorrect tool calls. Our code is available [here](#).

1 Introduction

Tool-calling language models increasingly use APIs, databases, and external systems to retrieve information or execute actions. Prior work such as Toolformer and ReAct shows that models can learn to use tools and interleave reasoning with actions (Schick et al., 2023; Yao et al., 2023). However, reliable tool use depends on a prior decision: whether a tool should be used at all.

This decision problem is not just a formatting issue. A syntactically valid tool call can still be wrong if the model hallucinates a missing argument or forces an irrelevant tool. We study this using When2Call, a benchmark that evaluates whether a model should call a tool, ask for missing information, or abstain when the available tools are insufficient (Ross et al., 2025). This setting is especially important for small open-weight models, which can benefit from external tools but may

be brittle in deciding when tool use is appropriate. It also raises a supervision problem: human labels for deployment-specific tool boundaries can be costly because annotators must inspect the user request, tool schemas, and required arguments.

Constitutional AI (CAI) offers one way to reduce this labeling burden. In a CAI-style pipeline, a written constitution specifies desired behavior, and a model uses those rules to critique, revise, or compare responses. In our setting, the constitution outlines when to call a tool, ask for missing information, or abstain. If the generated critiques and revisions point to the correct decision class, they can provide synthetic supervision for SFT and DPO.

As a running example, consider a customer support assistant connected to a proprietary weather API. The assistant should use the API when the user provides a location, ask a follow-up question when the location is missing, and avoid forcing a weather call when the user asks for something outside the tool’s scope. A CAI-style pipeline is appealing here because it could critique incorrect responses that fabricate locations or force irrelevant tool calls without requiring a human label for every example. We test whether this cheaper supervision preserves the intended decision boundary.

We ask two questions. First, which interventions most reliably improve observed three-way decision behavior? We compare zero-shot prompting, four-shot prompting, SFT, DPO, and CAI-style synthetic supervision. Second, when behavior fails, is the correct decision linearly recoverable from the model’s internal representation, or is it absent from that representation? We test this by training linear probes on hidden states at the final non-padding prompt token.

Our contributions are:

1. We evaluate tool-call decision making in small language models as a three-way decision problem among `tool_call`, `request_for_info`, and `cannot_answer`.
2. We compare prompting, SFT, DPO, and

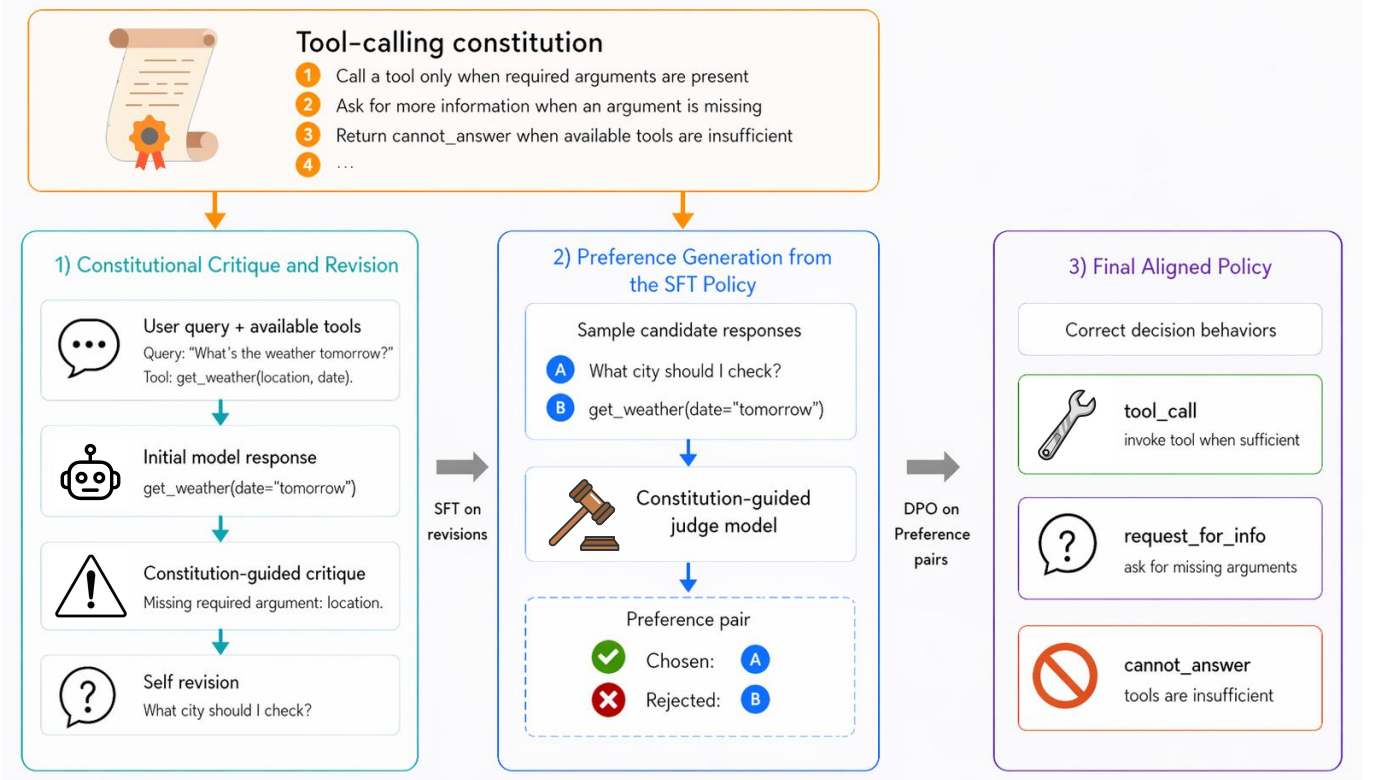


Figure 1: CAI-style synthetic supervision pipeline tested in this paper. A task-specific tool-calling constitution guides critique, revision, and preference generation, producing synthetic SFT targets and DPO preference pairs. We evaluate this pipeline as a hypothesis for improving three-way tool-call decisions, not as an assumed solution: the results show that its effectiveness depends on whether generated revisions and preferences preserve the intended decision behavior.

CAI-style synthetic supervision on this decision problem.

3. We audit CAI-generated revisions and preferences, showing that many fail to preserve the intended decision class and can collapse toward `tool_call`.
4. We use linear probes over hidden states to show that the correct decision is often linearly recoverable even when the final likelihood-ranked output selects the wrong action.

2 Background

2.1 Tool-call decision making

Tool-calling language models must decide not only how to call external tools, but whether tool use is necessary in the first place. Schick et al. (2023) show that language models can learn to use external APIs, and Yao et al. (2023) show that interleaving reasoning and actions can improve knowledge-intensive tasks. These systems demonstrate that tool use can improve language model behavior. Still, they do not fully isolate the decision of whether a model should call a tool, ask for

more information, or admit that the available tools are insufficient.

2.2 Dataset

When2Call directly studies this decision problem. Given a user request and a set of available tools, the benchmark evaluates whether a model chooses the appropriate tool-use behavior (Ross et al., 2025). In this paper, we focus on three target actions: `tool_call`, `request_for_info`, and `cannot_answer`. We use When2Call as the benchmark and a source of training examples for our contributions to the probing and Constitutional AI pipeline.

2.3 Prompting, post-training, and probing

We compare prompting and post-training because they represent different levels of intervention. Prompting tests whether a model can follow the tool-call decision rule from instructions and in-context examples without changing its weights (Brown et al., 2020). Supervised fine-tuning (SFT) tests whether labeled examples can directly

teach the desired action (Ouyang et al., 2022). Direct Preference Optimization (DPO) provides a simple preference-optimization objective over chosen/rejected response pairs, without training a separate reward model or running on-policy reinforcement learning (Rafailov et al., 2023). When2Call includes supervised and preference-training splits, which makes SFT and preference optimization natural post-training baselines for this setting (Ross et al., 2025).

Probing is used to test whether a model already encodes the information necessary to make a decision. In a standard probing setup, a lightweight classifier is trained on model representations to test whether a target property can be recovered from them (Belinkov, 2022). Recent work applies a similar idea to LLM reasoning, showing that hidden states can predict downstream generation outcomes before completion (Afzal et al., 2025). If the probe succeeds where generation fails, the model may encode the relevant decision signal but fail to express it during decoding. If both the probe and the generated decision fail, the decision boundary may not be well represented internally. We therefore use probing as a diagnostic rather than as a deployable tool-call policy: it tests whether behavioral failures reflect missing internal decision information or a failure to express information that is already recoverable.

2.4 Constitutional AI for tool-calling

In the original Constitutional AI setup by Anthropic, a model critiques and revises its own responses under a written constitution, then uses AI-generated preferences for reinforcement learning (RLAIF) (Bai et al., 2022). This structure makes CAI-style training a natural setting for tool-call decision making because the desired behavior can be expressed as a short set of rules: call a tool when the tool and arguments are sufficient, ask for missing information when the right tool exists but required arguments are missing, and say you cannot answer when the available tools are insufficient.

We adapt this idea to the When2Call setting by creating a task-specific constitution (set of decision rules for tool use). This lets us ask whether small models can produce useful self-critiques, revisions, and preference pairs given explicit behavioral rules.

3 Method

In our tool-call decision making setting, given a user request and a set of available tools, the model must choose among `tool_call`, `request_for_info`, and `cannot_answer`. We tackle this problem using four approaches. First, zero-shot and four-shot prompting test whether the model can follow the decision rule from instructions alone or from a small number of in-prompt examples respectively. Second, linear probing tests whether the correct decision is accessible from hidden states before generation. Third, SFT and DPO test whether standard post-training can improve the model’s decision behavior. Finally, a CAI-style pipeline tests whether explicit decision rules can generate useful self-critiques, revisions, and preference pairs for post-training.

Returning to our weather-assistant example, a fully specified query such as “What is the weather in College Park?” with an available weather API is labeled `tool_call`. An under-specified query such as “What is the weather there?” is labeled `request_for_info` because the required location argument is missing. A query outside the available tool set, such as legal contract advice when only a weather API is available, is labeled `cannot_answer`. Thus, the evaluation targets action selection rather than tool-call formatting alone.

3.1 Prompting Experiments

The prompting experiments test whether a model can follow the tool-call decision rule without updating its parameters. For each When2Call example, we render the user request and available tool specifications into a universal prompt (adjusted for model-specific formatting requirements). In the zero-shot condition, the prompt contains the task instruction, the user request, and the available tools. In the four-shot condition, we prepend four labeled demonstrations before the test example. The demonstrations cover the target decision behaviors so that the model sees examples of when to call a tool, when to ask for missing information, and when to return `cannot_answer`.

We include four-shot prompting as a baseline to test whether in-context learning is sufficient for this decision problem. If a small number of demonstrations can teach the decision boundary, then additional post-training may not be necessary. If four-shot prompting improves some classes but

leaves systematic errors, then model weights may need to be updated via SFT or DPO; or synthetic supervision methods such as CAI could be used. We do not treat these prompts as globally optimal; they are standardized prompting conditions used to compare model families and training interventions under a fixed evaluation setup. We provide full zero-shot and four-shot prompt templates in Appendix B.

3.2 Linear probe experiment

We test whether the correct tool-call decision is accessible from the model’s prompt representation before answer generation using linear probes. For each source example, we remove the final assistant response and render only the prompt-side context, which consists of the user request and available tools. We then freeze the language model, run the prompt through it, and extract hidden states at the final non-padding input token. Following the standard probing setup, we train lightweight classifiers on frozen model representations to test whether a target property can be recovered from them (Belingov, 2022). Here, the target property is the gold tool-call decision label.

We train a separate multinomial logistic-regression probe for each selected layer. Each probe predicts one of the three decision labels: `tool_call`, `request_for_info`, or `cannot_answer`. We train and evaluate probes for the prompted base models as well as the SFT and DPO baseline models, letting us test whether post-training changes the representation of the tool-call decision in the hidden states. Strong probe performance on the base models would suggest that the prompt representation separates the three decision labels without changing or improving the language model itself.

After training, we apply each probe to prompt-only hidden states from the When2Call test set. We compare probe accuracy against the model’s likelihood-based decision accuracy, and we also compare probe predictions directly to the model’s predictions. This lets us separate two failure modes: cases where the decision is not linearly recoverable from the prompt representation, and cases where the decision is recoverable but the model does not express it in its final prediction. Probe layer choices, feature extraction details, and classifier hyperparameters are provided in Appendix E.

3.3 CAI-style synthetic revision targets

We test whether explicit decision rules can produce reliable synthetic targets for small models using our CAI-style pipeline. Our constitution contains seven short rules that guide the model towards the correct decision behavior. We include the full constitution in Appendix C.

For CAI-SFT, each policy model first generates an initial response for a training prompt. The same model then receives the original prompt, the initial response, and the constitution, and produces a short critique of whether the response follows the constitution (tool-call decision rules). The model then receives all of the above plus the critique, and generates a revised response. The revised response becomes a synthetic supervised target for CAI-SFT. We do not assume these synthetic targets are correct, and later evaluate whether they preserve the intended decision class.

In the weather-assistant example, the constitution should make the critique reject fabricated locations for “What is the weather there tomorrow?” and reject irrelevant tool calls for legal-advice requests. The revised response should therefore ask for the missing location in the first case and return `cannot_answer` in the second.

We use the same model for self-critique and self-revision for both Gemma and Llama. This design keeps the supervision source internal to the model and deliberately tests whether a small model can use explicit rules to generate reliable feedback for its own tool-call decisions. We provide the full critique prompt, revision prompt, and preference judgment prompt in Appendix C.

3.4 CAI-guided preference generation

We use the next stage of CAI to produce preference pairs for DPO. Starting from the CAI-SFT policy, we sample two distinct candidate responses for each preference prompt. The base model (critic model) then receives the prompt, the two candidates, and the constitution, and selects the candidate that better follows the tool-call decision rules. The selected candidate becomes the chosen response, and the other becomes the rejected response. For example, for “What is the weather there tomorrow?”, a useful preference pair should prefer a follow-up question over a tool call with an invented location.

We also evaluate a CAI-DPO-from-base variant, where preference pairs are sampled from the

base model rather than from the CAI-SFT policy. This variant tests whether preference optimization is more useful before the policy has been shifted by synthetic SFT targets.

We train the final CAI-DPO policy on these constitution-guided preference pairs. We use DPO because the CAI judgment stage naturally produces chosen/rejected responses, and DPO directly optimizes from preference pairs without training a separate reward model or running on-policy reinforcement learning (Rafailov et al., 2023). When2Call uses an SPO variant for preference tuning, but our goal is not to compare preference objectives. We therefore use DPO to keep the comparison focused on the data-generation question: whether self-critique, self-revision, and self-judgment produce useful supervision.

4 Experimental Setup

4.1 Models and data

We run all experiments on Llama-3.2-3B-Instruct and Gemma-3-4B-it (Dubey et al., 2024; Gemma Team et al., 2025). All evaluations use the When2Call multiple-choice test split ($n = 3652$). Models choose among three supported actions: `tool_call`, `request_for_info`, and `cannot_answer`. The dataset also includes the label `direct`, but this class has zero gold support in the evaluation split. Therefore, any prediction of `direct` is unsupported by the gold labels in this evaluation split. We refer readers to Ross et al. (2025) for full dataset construction and evaluation details.

The public When2Call release includes separate supervised fine-tuning and preference-tuning training sets, which we use as the starting point for our post-training data construction. For most training and probe experiments, we construct a balanced three-way source corpus with equal counts of `tool_call`, `request_for_info`, and `cannot_answer`. This balanced corpus is used for standard SFT, CAI-SFT, CAI-DPO data generation, and probe training. The exception is the standard DPO baseline, which trains on the original When2Call preference split. Dataset construction, filtering rules, and class counts are listed in Appendix D.

4.2 Training conditions

For each model family, we evaluate:

- Zero-shot prompting

- Four-shot prompting
- Standard SFT
- Standard DPO
- CAI-SFT
- CAI-DPO from base
- CAI-DPO from CAI-SFT

Standard SFT trains on our balanced supervised source corpus, while standard DPO trains on the original When2Call preference split. CAI-SFT trains on constitution-guided revisions generated from the balanced CAI-SFT source split. CAI-DPO from base trains on constitution-guided preferences sampled from the base model, while CAI-DPO from CAI-SFT trains on constitution-guided preferences sampled from the CAI-SFT policy.

All SFT and DPO runs use the same LoRA configuration and optimization settings across baseline and CAI variants; only the training data and, for CAI-DPO from CAI-SFT, the starting checkpoint differ (Hu et al., 2022; Dettmers et al., 2023). We include our full training setup in Appendix F.

4.3 Probe setup

We train multinomial logistic-regression probes for the prompted base models and for the SFT and DPO checkpoints. For a model with L transformer layers, we probe the middle layer $\lceil 0.5L \rceil$, a late layer $\lceil 0.75L \rceil$, and the final layer L . Each probe uses the hidden state at the final non-padding prompt token and predicts one of the three decision labels. Train/dev splits, feature standardization, regularization settings, and feature-extraction details are provided in Appendix E.

4.4 Evaluation metrics

For behavioral evaluation, we use the When2Call likelihood-based MCQ evaluator. For each test example, we score each candidate decision option as a continuation and select the option with the highest score. We report byte-length-normalized log-probability because candidate strings differ in length. Raw scores are reported in our Appendices. We report accuracy, macro-F1, class-wise precision, class-wise recall, class-wise F1, and prediction distributions. For probes, we compare probe predictions with gold labels and with the corresponding model’s normalized MCQ predictions. As a robustness check, we also compute one-vs-rest area under the receiver operating characteristic curve (AUROC) for the linear probes to see whether each decision class is separable from the others in the hidden-state representations.

Method	Accuracy	Macro-F1
Llama-3.2-3B-Instruct		
ZS	0.471	0.307
4-shot	0.590	0.432
SFT	0.799	0.590
DPO	0.585	0.418
CAI-SFT	0.479	0.307
CAI-DPO (Base)	0.475	0.311
CAI-DPO (SFT)	0.482	0.310
Gemma-3-4B-it		
ZS	0.280	0.237
4-shot	0.274	0.235
SFT	0.282	0.235
DPO	0.287	0.242
CAI-SFT	0.402	0.211
CAI-DPO (Base)	0.523	0.352
CAI-DPO (SFT)	0.402	0.210

Table 1: Primary byte-length-normalized accuracy and macro-F1 on the When2Call test set. Standard SFT is the strongest method for Llama-3.2-3B-Instruct. For Gemma-3-4B-it, CAI-DPO from the base model achieves the best normalized accuracy and macro-F1.

5 Results

We report our full suite of results in Appendices A.1–A.5.

5.1 SFT is strongest for Llama, while CAI-DPO from base helps Gemma

Table 1 shows our main behavioral results.

For Llama, SFT produces the best accuracy and macro-F1. Four-shot prompting and DPO perform similarly, suggesting that in-context demonstrations already partially recover the decision boundary. In contrast, the Llama CAI variants remain close to the zero-shot baseline.

For Gemma, CAI-DPO from the base model is the strongest setting, raising accuracy from 0.280 under zero-shot prompting to 0.523 and macro-F1 from 0.237 to 0.352. CAI-SFT also improves accuracy to 0.402 but lowers macro-F1 to 0.211. CAI-DPO from CAI-SFT is nearly identical to CAI-SFT, with 0.402 accuracy and 0.210 macro-F1, indicating that preference optimization from the already-skewed CAI-SFT policy does not recover balanced behavior.

Overall, Table 1 shows that CAI-style supervision is not uniformly stronger than standard post-

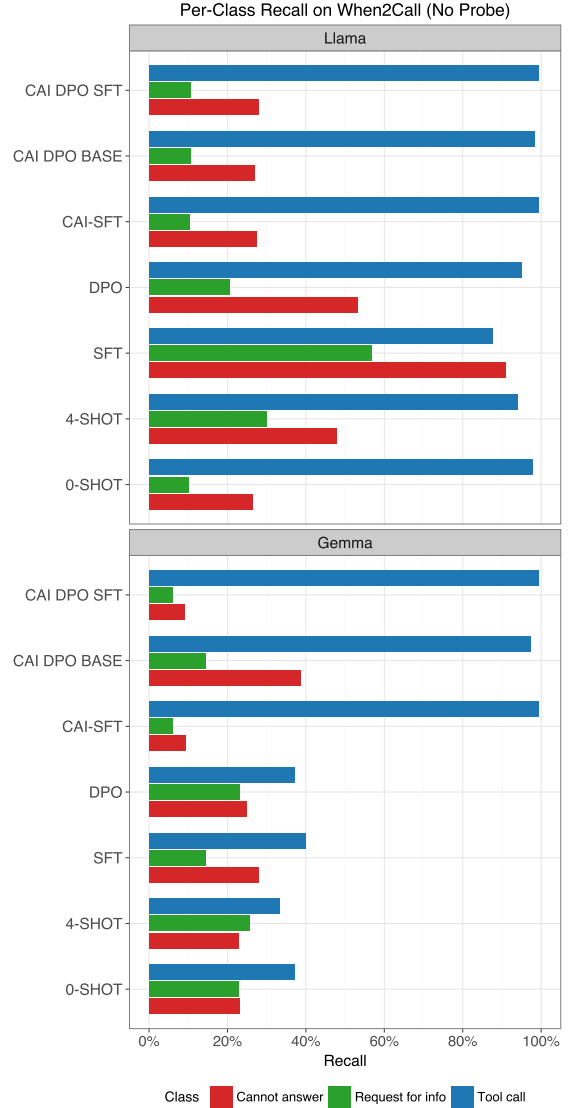


Figure 2: CAI improves `tool_call` recall more reliably than the two non-tool-call classes. For Llama, standard SFT remains the most balanced condition. For Gemma, CAI-DPO from the base model improves over CAI-SFT, while CAI-DPO from CAI-SFT remains close to the CAI-SFT collapse pattern. Overall, the strongest CAI gains still concentrate on `tool_call` behavior rather than uniformly improving all decision classes.

training. It helps Gemma most clearly in the CAI-DPO-from-base branch, but the other CAI variants remain close to prompting-level behavior.

5.2 CAI gains are concentrated in `tool_call` behavior

Figure 2 explains why CAI can improve accuracy without improving macro-F1. For Llama, standard SFT gives the strongest recall on the two non-tool-call classes, reaching 0.569 recall

Intended class	Llama	Gemma
<code>request_for_info</code>	5/1500	3/1500
<code>cannot_answer</code>	63/1500	65/1500

Table 2: CAI-SFT revisions that preserve the intended non-tool-call class. Most revised outputs for `request_for_info` and `cannot_answer` examples become tool-call-shaped supervision.

for `request_for_info` and 0.910 recall for `cannot_answer`. CAI variants instead drive `tool_call` recall close to 1.0 while leaving the other two classes near prompting-baseline levels.

For Gemma, CAI-DPO from the base model is the only CAI variant that substantially improves balanced performance. Compared with CAI-SFT, it preserves high `tool_call` recall while improving recall for `request_for_info` and `cannot_answer`. This explains why Gemma CAI-DPO (Base) improves both accuracy and macro-F1 relative to CAI-SFT. In contrast, Gemma CAI-DPO from CAI-SFT remains nearly identical to CAI-SFT, with `tool_call` recall at 0.992 and very low recall for `request_for_info` and `cannot_answer`.

The prediction distributions in Appendix A.4 show the same pattern from another angle: Gemma CAI-SFT reduces unsupported `direct` predictions from 0.267 under zero-shot prompting to 0.042, but shifts 0.882 of all predictions to `tool_call`. Thus, CAI can remove one failure mode while introducing another.

5.3 CAI supervision often corrupts the intended decision class

The `tool_call` shift is already visible in the generated CAI supervision. Although the CAI-SFT source split is balanced by intended decision class, the revised outputs often do not preserve that class. Table 2 shows that only 5/1500 Llama `request_for_info` revisions and 63/1500 Llama `cannot_answer` revisions match their intended class; Gemma shows the same pattern, with 3/1500 and 65/1500 matching revisions, respectively.

This helps explain why CAI-SFT and CAI-DPO-from-CAI-SFT remain highly skewed toward `tool_call`: many examples intended to teach `request_for_info` or `cannot_answer` behavior instead become additional tool-call supervision.

5.4 Linear probes reveal a representation–behavior gap

Table 3 compares behavioral MCQ performance with the best linear probe for each condition. *Probe Layer* reports the layer with the highest probe accuracy with respect to the gold When2Call response, while *Agree. Layer* reports the layer with the highest agreement between the probe and the model’s normalized MCQ prediction. Layer labels refer to the three probed depths: middle layer, 75%-depth layer, and final layer.

For Gemma, probe accuracy substantially exceeds model accuracy across zero-shot, SFT, and DPO checkpoints. This suggests that the three-way decision is more recoverable from hidden states than from the model’s final likelihood-ranked output.

For Llama, the pattern is more mixed. In the zero-shot, four-shot, and DPO settings, probe accuracy exceeds behavioral accuracy, suggesting that some decision information is linearly recoverable before generation. Under SFT, however, behavioral accuracy exceeds probe accuracy, indicating that supervised training improves the expressed decision policy beyond what is captured by this simple probe comparison.

Probe-model agreement provides a second diagnostic: it measures whether the linearly decoded decision is the same decision favored by the model’s likelihood-ranked output. Agreement remains low for Gemma even when probe accuracy is high, especially for Gemma SFT. This indicates that the probe is not merely reproducing the model’s behavioral bias; it often recovers a different, more ground-truth aligned decision boundary from the hidden state. Class-wise probe precision, recall, and F1 are reported in Appendix A.2.

Per-class AUROC shows that `cannot_answer` is often more separable than `request_for_info`, suggesting that missing-argument cases remain a harder decision boundary. Full AUROC results are reported in Appendix A.3.

6 Discussion

The results suggest that tool-call decision making depends on both internal representation and output behavior. The probe results show that the three-way decision is often recoverable from prompt-side hidden states even when the model’s likelihood-ranked output is wrong. This gap is

Model	Condition	Model Acc.	Model F1	Probe Acc.	Probe F1	Probe Layer	Agree.	Agree. Layer
Llama-3.2-3B	ZS	0.471	0.307	0.547	0.550	mid	0.381	75%
Llama-3.2-3B	4-shot	0.590	0.432	0.621	0.623	mid	0.529	mid
Llama-3.2-3B	SFT	0.799	0.590	0.614	0.606	75%	0.616	75%
Llama-3.2-3B	DPO	0.585	0.418	0.553	0.546	final	0.542	final
Gemma-3-4B	ZS	0.280	0.237	0.487	0.483	75%	0.297	final
Gemma-3-4B	4-shot	0.274	0.235	0.557	0.558	mid	0.281	final
Gemma-3-4B	SFT	0.282	0.235	0.612	0.612	75%	0.285	75%
Gemma-3-4B	DPO	0.287	0.242	0.513	0.506	final	0.307	final

Table 3: Linear probes often recover the tool-call decision better than the model expresses it behaviorally. Probe Layer reports the probed layer with the highest probe accuracy for each condition, and Agree. Layer reports the probed layer with the highest agreement between probe predictions and the model’s predictions. Layer labels correspond to the three probed depths: middle layer ($\lceil 0.5L \rceil$), late layer ($\lceil 0.75L \rceil$), and final layer (L).

most visible for Gemma: its non-CAI behavioral accuracy remains near the zero-shot baseline, while its probes recover substantially more decision information. This should not be interpreted as causal evidence that the model “knows” the correct action, but it does suggest that some failures still arise after decision-relevant information is already present in the hidden state.

The probe mismatches also show that the representation-behavior gap differs by model family. For Llama, many probe-correct/model-wrong cases involve `request_for_info` examples that the model behaviorally scores as `tool_call`. In these cases, the linear probe often recovers the `request_for_info` decision, but the model’s output still favors tool use. The per-class AUROC results in Appendix A.3 are consistent with this pattern: `request_for_info` is generally less separable than `cannot_answer`, suggesting that the `request_for_info` cases remain a harder boundary. For Gemma, the mismatch is broader: probe-correct/model-wrong examples often involve unsupported `tool_call` predictions, especially when the gold label is `cannot_answer`. Thus, Gemma’s poor behavioral performance reflects a broad output selection mismatch: decision information is often recoverable from hidden states, but the final scored action still selects the wrong behavior.

Llama responds to post-training more effectively. Standard SFT is the strongest overall method for Llama, while four-shot prompting and DPO provide smaller gains. This suggests that, for Llama, direct supervised targets align the model’s final action policy better than prompting or pairwise preference learning alone. Gemma shows the opposite pattern: standard SFT and DPO barely

improve behavior despite stronger probe performance. One possible explanation is Gemma’s training history. Gemma 3 uses distillation from a larger instruct model as part of its post-training recipe (Gemma Team et al., 2025). Distillation can compress strong teacher behavior into a smaller model, but it may also produce a more rigid instruction following policy that is harder to reshape with small SFT or DPO runs. We treat this as a hypothesis rather than a causal explanation.

The CAI results are best explained by the quality of the generated supervision. As shown in Section 5.3, many CAI-SFT revisions for `request_for_info` and `cannot_answer` examples do not preserve the intended decision class. Table 4 shows representative failure cases. The common pattern is that the pipeline treats tool-call plausibility as a proxy for correctness: missing arguments are filled with fabricated values, insufficient tools are treated as if they were adequate, and invalid tool-call formats are sometimes preferred over correct abstention behavior. In the running weather-assistant example, this is the difference between asking for the missing location and fabricating one in order to produce a plausible call to the weather tool.

These failures suggest that the critique stage often checks surface level validity rather than the central decision boundary among calling a tool, asking for missing information, and declining because the tools are insufficient. This explains why CAI-style supervision can improve `tool_call` behavior while failing to improve balanced three-way decision making.

The revision stage amplifies this problem because the CAI prompt is substantially more complex than the original decision prompt: the model must condition on the user request, tool schema,

Failure mode	Stage	Prompt type	Pipeline behavior	Interpretation
Missing argument	Critique / revision	A coordinate query provides longitude but omits latitude.	The model calls the coordinate tool with an invented latitude; in some cases, the critic marks the response as issue-free.	A <code>request_for_info</code> case is converted into a tool call because the critic fails to enforce required-argument availability.
Insufficient tool converted to invented tool	Revision	A request asks for a linear-regression prediction, but no available tool can perform linear regression.	A correct cannot-answer-style response is revised into a call to a nonexistent <code>linear_regression</code> tool.	The critique/revision stage treats a tool description as evidence that the tool can satisfy the request, leading the model to invent a tool-call response.
Cannot-answer expressed as fake tool call	Preference generation / judgment	A stock-screening request cannot be satisfied by the available tools.	DPO candidates include invalid tool calls such as <code>no_available_tools</code> or placeholder tool names; the judge prefers the less bad one.	The model recognizes insufficiency but expresses it through tool-call format, corrupting the target action class.
Both-wrong preference pair	Preference generation / judgment	An air-quality forecast request names a city but omits required location arguments.	Both DPO candidates call the forecast tool with fabricated coordinates, and the judge selects one as being more “direct.”	DPO receives a preference between two wrong tool calls, so the chosen response is not reliable positive supervision.

Table 4: Representative CAI failure cases from the manual audit, organized by the stage at which the error enters the synthetic-supervision pipeline. The dominant failure is not random noise: critique, revision, and preference generation often convert missing-information or insufficient-tool cases into plausible-looking tool calls.

constitution, initial response, and critique. In this setting, small models may treat revision as producing a more plausible tool call rather than preserving the intended decision class. This is especially harmful when the initial response is already a tool call, because the easiest revision is to preserve or minimally edit the tool-call format.

The DPO stage is also sensitive to preference pair quality. DPO assumes that the chosen response is meaningfully better than the rejected response (Rafailov et al., 2023). In our CAI pipeline, some sampled pairs were duplicates, same-class pairs, or pairs where both candidates were behaviorally wrong. The judge may still select one because it is shorter, more direct, or more consistent with the expected tool-call schema, rather than because it expresses the correct decision. This helps explain why CAI-DPO-from-CAI-SFT remains close to CAI-SFT, while CAI-DPO-from-base performs better because its candidate distribution is less collapsed.

The probe and CAI analyses therefore diagnose different parts of the problem. We did not probe the CAI checkpoints, so the probe results should not be read as a direct explanation of CAI behavior. Instead, probing shows that non-CAI checkpoints can contain recoverable action information

that is not realized in the final likelihood-ranked output, while the CAI audit shows that it can fail when synthetic supervision is not decision-faithful. Together, these findings suggest that improving tool-call decision making requires both better alignment between internal decision signals and output behavior, and stricter verification that generated training targets preserve the intended action class. Our analysis does not rule out CAI as a general approach. It shows that for small models, in this setting, CAI-style tool-call training needs stronger verification, cleaner counterfactual preference pairs, or a simpler decision-only supervision format.

7 Related Work

7.1 Tool-use benchmarks and agent evaluation

Tool-use evaluation has expanded from basic API calling ability to broader agentic behavior. The Berkeley Function Calling Leaderboard (BFCL) evaluates whether models can produce correct function calls across increasingly complex tool-use settings (Patil et al., 2025). MetaTool focuses more directly on whether a model should use a tool and which tool it should choose (Huang et al., 2023). These benchmarks are related to our

setting, but our work focuses specifically on the decision boundary among calling a tool, asking for more information, and declining because the tools are insufficient. This makes our work closer to decision calibration than to schema correctness alone.

7.2 Selective prediction and abstention

Tool-call decision making is related to selective prediction and abstention because the model must sometimes avoid answering. Prior work on abstention studies when NLP systems should withhold predictions instead of making likely errors (Xin et al., 2021). Our setting differs because abstention is not the only non-answer behavior. The model must also distinguish between asking the user for missing information and saying that no available tool can satisfy the request. This makes the task a structured action-selection problem rather than a binary answer-versus-abstain decision.

7.3 Model-internal signals for tool use and confidence

Recent work has used internal representations to estimate whether an LLM response is likely to succeed. Model-internal confidence estimation for tool-using agents is especially close to our probe analysis because it uses internal model signals to calibrate agent behavior under tool-use risk (Subramani et al., 2025).

7.4 Self-feedback and synthetic supervision

Our CAI pipeline also connects to self-feedback methods such as Self-Refine, where models iteratively critique and revise their own outputs (Madaan et al., 2023). The difference is that our feedback is constrained by a task-specific tool-calling constitution rather than a general instruction to improve the answer. This matters because the target behavior is not open-ended quality improvement; it is a constrained decision about whether the tool set is sufficient, incomplete, or actionable. Our results further suggest that, in this kind of constrained action-selection task, self-feedback must preserve the intended decision class rather than merely produce a more fluent or tool-call-like response.

8 Conclusion and Future Work

We study tool-call decision making in small language models, focusing on whether models

should call a tool, ask for missing information, or say they cannot answer. Across Gemma-3-4B-it and Llama-3.2-3B-Instruct, standard SFT produces the strongest balanced Llama behavior, while CAI-DPO from the base model produces the strongest Gemma result. However, CAI-SFT and CAI-DPO from CAI-SFT show that constitution-guided training can still shift behavior unevenly toward `tool_call` when the generated supervision is not decision-faithful. The probe results add a separate diagnostic finding: in non-CAI checkpoints, the correct decision is often recoverable from prompt-side hidden states even when the model does not express that decision in its final likelihood-ranked output.

Together, these results suggest two distinct failure points for small-model tool-call decision making. First, some errors arise because recoverable decision information is not reliably converted into the final scored action. Second, post-training can fail or even reinforce the wrong behavior when synthetic revisions and preference pairs do not preserve the intended action class.

One direct extension is to keep the CAI structure but add stronger verification. A stronger external critic or rule-based verifier could check whether required arguments are present, whether the selected tool can actually satisfy the request, and whether the response invents tools or placeholder arguments (Bai et al., 2022).

Future work should also test prompt ablations and decision-only CAI supervision. Rather than asking a small model to generate full tool-use responses under a long constitution, the feedback stage could supervise only one of the three action labels. A separate model could then generate the tool call only when the decision model predicts `tool_call`. This would separate the decision problem from the tool-call formatting problem and make the constitutional feedback easier to verify. For preference learning, future work should filter DPO pairs by absolute correctness, keeping only pairs where exactly one candidate preserves the intended decision class.

Another direction is to formulate tool-call decision making as a contextual bandit problem, where the user request and available tools define the context, the model selects a decision action, and feedback rewards whether that action is appropriate. This could provide a cleaner objective for improving action selection than CAI-style self-revision.

For probing, future work should go beyond measuring whether the decision is decodable and study why recoverable decision signals do not always control the final model output, for example, through causal interventions on the hidden states.

9 Limitations

This work studies tool-call decision making as a classification problem. We do not evaluate whether a generated tool call is valid, executable, or semantically correct. For example, a model can receive credit for choosing the `tool_call` action even if the actual function name, argument values, or schema formatting would be wrong in a real tool-use system. This follows our focus on the When2Call decision setting, but it means our results should not be interpreted as measuring end-to-end tool-use correctness.

Our experiments are limited to two small instruction-tuned model families. Anthropic originally developed Constitutional AI for a much more capable assistant model, where the model is expected to critique, revise, and provide feedback under a written constitution. In our setting, the small policy model must do those same steps while also tracking the user request, tool schema, constitution, initial response, and critique. This makes our results a small-model stress test of CAI-style training, not a general claim about how Constitutional AI would behave with stronger frontier models.

The CAI results reflect one specific pipeline design, not a complete test of Constitutional AI for tool use. In our setup, the same small model family critiques, revises, and judges its own outputs. Our audit shows that these generated revisions and preference pairs often fail to preserve the intended behavior class, especially for `request_for_info` and `cannot_answer` examples. Stronger external critics, stricter filtering of generated targets, absolute correctness checks before DPO, or a simpler decision-only output format could produce different results.

Our results also depend on the prompt templates used for critique, revision, and preference judgment. We include the full templates in the appendices for reproducibility, but we do not claim they are prompt-optimal. Different wording, explicit argument-presence checks, or clearer separation between action selection and response formatting could change the observed behavior.

Linear probes show that the decision label is recoverable from hidden states, but they do not prove that the model causally uses that information when generating or scoring outputs. We also did not probe the CAI checkpoints, so the probe analysis should be interpreted as a diagnostic of the prompting and standard post-training checkpoints rather than as a direct explanation of CAI behavior. Establishing a causal relationship would require intervention-based experiments beyond the scope of this paper.

Finally, our evaluation is benchmark-based and mostly single-turn. Real tool-integrated systems often require multi-turn clarification, execution feedback, error handling, and tool-result interpretation. Our results therefore isolate an important decision stage, but they do not measure full agent reliability in deployed tool-use environments.

References

- Afzal, A., Matthes, F., Chechik, G., and Ziser, Y. (2025). Knowing before saying: LLM representations encode information about chain-of-thought success before completion. In *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria.
- Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McInnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, T. B. B., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Lukosiute, K., Askell, et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.
- Belinkov, Y. (2022). Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., and Hesse, C. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*.
- Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., Fan, A., et al. (2024). The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., Grill, J.-B., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., et al. (2025). Gemma 3 technical report. Technical report, Google DeepMind. *arXiv:2503.19786*.

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- Huang, Y., Shi, J., Li, Y., Fan, C., Wu, S., Zhang, Q., Liu, Y., Zhou, P., Wan, Y., Gong, N. Z., and Sun, L. (2023). MetaTool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744.
- Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. (2025). The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741.
- Ross, H., Mahabaleshwarkar, A. S., and Suhara, Y. (2025). When2Call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Albuquerque, New Mexico.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.
- Subramani, N., Eisner, J., Svegliato, J., Van Durme, B., Su, Y., and Thomson, S. (2025). MICE for CATs: Model-internal confidence estimation for calibrating agents with tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Albuquerque, New Mexico.
- Xin, J., Tang, R., Yu, Y., and Lin, J. (2021). The art of abstention: Selective prediction and error regularization for natural language processing. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

A Additional Results

A.1 Class-wise behavior metrics

Table 5 reports class-wise precision, recall, and F1 for the behavioral prompting and post-training runs under normalized scoring. TC denotes tool_call, RI denotes request_for_info, and CA denotes cannot_answer.

A.2 Class-wise probe metrics

Table 6 reports class-wise precision, recall, and F1 for the best probe layer for each probe setup. Probe rows are evaluated against the same three target labels as the behavioral runs.

A.3 Probe AUROC robustness check

We report one-vs-rest AUROC as a robustness check for the probe results. AUROC tests whether the probe ranks the gold decision class above alternatives, which directly measures class separability in the hidden states. This complements accuracy and macro-F1, which evaluate only the final discrete probe prediction.

A.4 Prediction distributions

Table 7 reports the normalized prediction distribution across candidate labels. The direct label has zero gold support in the evaluation split, so direct predictions are unsupported in this setting.

A.5 Byte-normalization effect

Table 8 reports the effect of byte-length normalization relative to raw summed log-probability scoring. Positive deltas mean normalized scoring increased the metric relative to raw scoring.

B Prompt Templates and Few-Shot Examples

B.1 Zero-shot prompt template

Llama family. The zero-shot Llama prompt used the following system message and user turn format:

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
↪ _ider_id|>
You are an expert in composing functions. You are
↪ given a question and a set of possible
↪ functions.
Based on the question, you will need to make one or
↪ more function/tool calls to achieve the
↪ purpose.
If none of the functions can be used, point it out.
If the given question lacks the parameters required
↪ by the function, also point it out.
You should only return the function call in tool
↪ call sections.
If you decide to invoke any of the function(s), you
↪ MUST put it in the format of
[func_name1(param1=value1, param2=value2...),
↪ func_name2(...)]
You SHOULD NOT include any other text in the
↪ response if you call a function.
<|eot_id|>
<|start_header_id|>user<|end_header_id|>
Here is a list of functions in JSON format that you
↪ can invoke.
<TOOLS_JSON>

<QUESTION>
<|eot_id|><|start_header_id|>assistant<|end_header_id|>
↪ _id|>
```

Gemma family. The zero-shot Gemma prompt used the following user turn format:

```
<start_of_turn>user
You have access to functions.

Decide how to respond to the question using the
↪ provided functions.
- If a provided function directly answers the
↪ question and the required arguments are
↪ available, return a tool call.
- If the correct function exists but a required
↪ argument is missing, ask for that missing
↪ information.
- If no provided function can answer the question,
↪ say that you cannot answer with the provided
↪ tools.

If you decide to invoke a function, you MUST put it
↪ in the format:
{"name": "tool_name", "arguments": {"argument1":
↪ "value1", "argument2": "value2"}}

You SHOULD NOT include any other text in the
↪ response if you call a function.

Functions:
<TOOLS_JSON>
Question:
<QUESTION><end_of_turn>
<start_of_turn>model
```

B.2 Four-shot prompt template

The four-shot prompts used the same model-family-specific templates as the zero-shot prompts, with four fixed demonstrations inserted before the final test question. For Llama, demonstrations were formatted as user-assistant exchanges. For Gemma, demonstrations were formatted as inline blocks with Functions, Question, and Answer fields.

The four frozen demonstrations were:

1. Tool-call example.

Question: “Show me completed ICOs in the e-commerce and finance sectors, sorted by funds raised, with a UTC offset of 8 hours.”

Tools:

```
get_ico_calendar(category, time_utc_offset,
↪ tabname, sort, lang_id)
```

Target answer:

```
<TOOLCALL>[{"name": "get_ico_calendar", "arguments":
↪ ": {
  "category": "_ico_cat_ecomm, _ico_cat_finance",
  "time_utc_offset": 28800,
  "tabname": "completed",
  "sort": "funds_raised"}]}]</TOOLCALL>
```

2. Request-for-information example.

Question: “I am interested in finding some radio channels that have ‘pop’ in their names. Can you help me with that?”

Tools:

```
get_channels(keyword, genre_id, city_id,
↪ country_id, page)
Required: keyword, country_id, page
```

Target answer:

To assist you better, could you please specify
↪ the page number for the search results?

3. Tool-call example.

Question: “Convert RGB color code 255, 0, 0 to CMYK.”

Tools:

```
rgb_to_cmyk(r, g, b)
generate_qr(url)
```

Target answer:

```
<TOOLCALL>[{"name": "rgb_to_cmyk", "arguments": {
  "r": 255,
  "g": 0,
  "b": 0}}]</TOOLCALL>
```

4. Cannot-answer example.

Model	Method	TC-P	TC-R	TC-F1	RI-P	RI-R	RI-F1	CA-P	CA-R	CA-F1
Llama	ZS	0.497	0.979	0.659	0.406	0.102	0.163	0.879	0.265	0.407
	4-shot	0.586	0.939	0.721	0.533	0.300	0.384	0.891	0.479	0.623
	SFT	0.754	0.876	0.811	0.831	0.569	0.675	0.840	0.910	0.874
	DPO	0.545	0.949	0.692	0.626	0.206	0.310	0.903	0.532	0.670
	CAI-SFT	0.468	0.994	0.636	0.484	0.103	0.169	0.915	0.274	0.422
	CAI-DPO (Base)	0.498	0.982	0.661	0.409	0.105	0.168	0.893	0.269	0.414
	CAI-DPO (SFT)	0.469	0.994	0.637	0.496	0.105	0.174	0.914	0.279	0.427
Gemma	ZS	0.408	0.372	0.389	0.311	0.228	0.263	0.415	0.231	0.297
	4-shot	0.402	0.333	0.364	0.320	0.255	0.284	0.407	0.229	0.293
	SFT	0.393	0.399	0.396	0.381	0.144	0.209	0.418	0.279	0.334
	DPO	0.421	0.370	0.394	0.298	0.231	0.260	0.426	0.249	0.315
	CAI-SFT	0.399	0.992	0.569	0.418	0.060	0.105	0.960	0.092	0.168
	CAI-DPO (Base)	0.492	0.972	0.653	0.414	0.143	0.213	0.907	0.386	0.542
	CAI-DPO (SFT)	0.398	0.992	0.568	0.433	0.061	0.107	0.959	0.090	0.165

Table 5: Class-wise behavioral metrics under normalized scoring for prompting and post-training runs.

Model	Probe Setup	TC-P	TC-R	TC-F1	RI-P	RI-R	RI-F1	CA-P	CA-R	CA-F1
Llama	ZS	0.575	0.461	0.512	0.408	0.602	0.486	0.729	0.589	0.652
	4-shot	0.565	0.665	0.611	0.513	0.597	0.552	0.868	0.598	0.708
	SFT	0.541	0.780	0.639	0.539	0.464	0.499	0.846	0.570	0.681
	DPO	0.496	0.619	0.551	0.451	0.396	0.422	0.723	0.615	0.665
Gemma	ZS	0.472	0.494	0.483	0.380	0.393	0.386	0.602	0.558	0.579
	4-shot	0.602	0.453	0.517	0.435	0.685	0.533	0.714	0.554	0.624
	SFT	0.618	0.619	0.618	0.490	0.604	0.541	0.757	0.612	0.677
	DPO	0.478	0.531	0.503	0.416	0.387	0.401	0.631	0.598	0.614

Table 6: Class-wise probe metrics for the best probe layer in each setup.

Question: “Find the 99% confidence interval for a sample with a standard deviation of 3, mean of 12, and size of 40.”

Tools:

```
monthly_mortgage_payment(loan_amount,
↪ annual_interest_rate, loan_term_years)
cosine_similarity(vector_a, vector_b)
```

Target answer:

Apologies, but I'm unable to calculate the 99%
↪ confidence interval.
This task requires statistical analysis that I
↪ currently cannot perform.

In the actual prompts, the full serialized JSON tool schemas from the released `fewshot_examples.template.json` file were included verbatim for each demonstration.

C Constitution and CAI Prompts

C.1 Tool-calling constitution

The CAI pipeline used the following seven-rule tool-calling constitution:

Tool-Calling Constitution

- Responses must be either a tool call, follow-up
↪ question, or cannot-answer response.
- Use a tool only when an available tool can solve
↪ the request and all required arguments are
↪ present.
- If the right tool exists but required
↪ information is missing, ask a short follow-up
↪ only for the missing required fields.
- If no available tool can solve the request, or
↪ if no tools are available, give a cannot-answer
↪ response.
- Never invent tools, arguments, facts, IDs,
↪ dates, or other missing fields.
- Do not ask for optional information or for
↪ information the user already provided.
- When using a tool, use the correct tool, only
↪ supported arguments, and the required format.

C.2 Self-critique prompt

The critique prompt was:

You are checking an assistant response against the
↪ Tool-Calling Constitution.

Constitution:
{{CONSTITUTION}}

User Request:
{{USER_REQUEST}}

Available Tools:
{{TOOLS}}

Assistant Response:
{{ASSISTANT_RESPONSE}}

Decide whether the response violates the
↪ constitution. Focus on the single most
↪ important issue, if any.
If no tools are available, any tool call is a
↪ violation and the correct behavior is to
↪ explain why you cannot answer.

Allowed issue labels for `Primary Issue` when there
↪ are issues:

- should_have_called_tool
- missing_required_information
- tools_insufficient
- invented_tool_or_arguments
- direct_answer_when_tool_required
- wrong_tool
- unsupported_arguments
- bad_tool_call_format

Use exactly one label from the list above.
Do not repeat the full list in the output.

Output exactly:
Verdict: NO_ISSUES
Primary Issue: none
Critique: No issues

or

Verdict: ISSUES
Primary Issue: wrong_tool
Critique: <one short sentence>

C.3 Self-revision prompt

The revision prompt was:

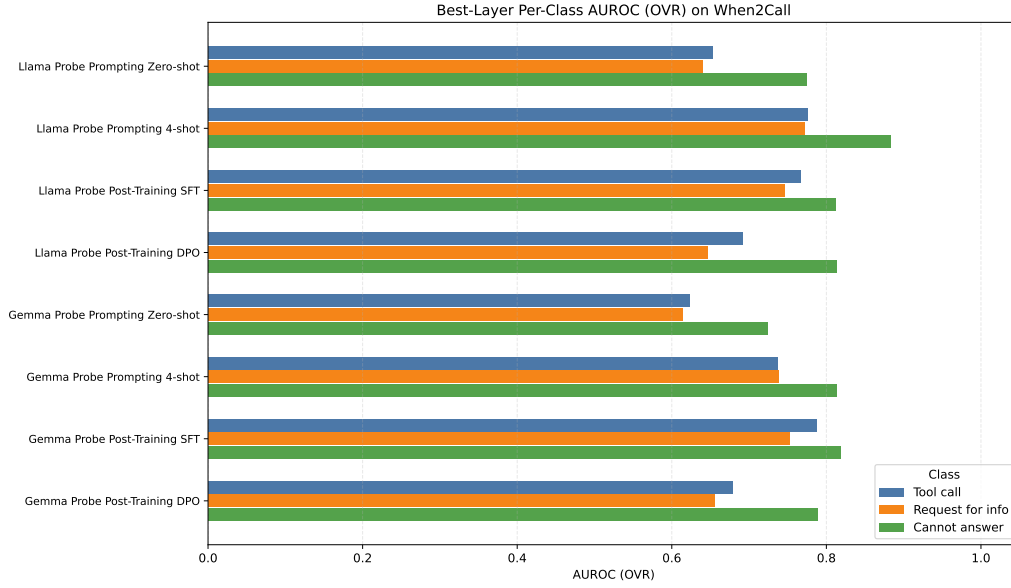


Figure 3: Best-layer per-class one-vs-rest AUROC for linear probes on When2Call. The results show that `cannot_answer` is often more separable than `request_for_info`, consistent with the difficulty of distinguishing missing-argument cases from valid tool-call cases.

Model	Method	Direct	Tool Call	Req. Info	Cannot Ans.
Llama	ZS	0.122	0.699	0.073	0.107
	4-shot	0.077	0.568	0.164	0.191
	SFT	0.005	0.412	0.199	0.384
	DPO	0.077	0.618	0.096	0.209
	CAI-SFT	0.079	0.753	0.062	0.106
	CAI-DPO (Base)	0.119	0.699	0.075	0.107
	CAI-DPO (SFT)	0.079	0.751	0.062	0.108
Gemma	ZS	0.267	0.323	0.213	0.197
	4-shot	0.274	0.294	0.232	0.200
	SFT	0.293	0.360	0.110	0.237
	DPO	0.256	0.311	0.225	0.208
	CAI-SFT	0.042	0.882	0.042	0.034
	CAI-DPO (Base)	0.047	0.701	0.100	0.151
	CAI-DPO (SFT)	0.042	0.884	0.041	0.033

Table 7: Prediction distributions across candidate decision labels under normalized scoring. These distributions show whether gains come from balanced decision improvement or from shifting probability mass toward a dominant label.

```

You are revising an assistant response to follow
↪ the Tool-Calling Constitution.

Constitution:
{{CONSTITUTION}}

User Request:
{{USER_REQUEST}}

Available Tools:
{{TOOLS}}

Original Response:
{{ASSISTANT_RESPONSE}}

Critique:
{{CRITIQUE}}

Rewrite the response so it follows the constitution.

If the critique is invalid, contradictory, or does
↪ not make sense given the user request,
↪ available tools, and original response, ignore
↪ the critique and preserve the original
↪ response.

```

```

If the original response already follows the
↪ constitution better than the critique suggests,
↪ reproduce the original response exactly.
Never change a correct response into a worse or
↪ malformed one.
If no tools are available, do not output a tool
↪ call; rewrite the response explaining why you
↪ cannot answer.

If the correct output is a tool call, use exactly:
<TOOLCALL>{"name":"tool_name","arguments":{...}}</>
↪ TOOLCALL>

Output only the revised response.

```

C.4 Preference judgment prompt

The preference prompt was:

```

You are choosing the better assistant response
↪ under the Tool-Calling Constitution.

Constitution:
{{CONSTITUTION}}

User Request:

```

Model	Method	Raw Acc.	Norm. Acc.	Δ Acc.	Raw F1	Norm. F1	Δ F1
Llama	ZS	0.377	0.471	+0.093	0.173	0.307	+0.135
	4-shot	0.445	0.590	+0.145	0.268	0.432	+0.164
	SFT	0.740	0.799	+0.059	0.545	0.590	+0.045
	DPO	0.432	0.585	+0.153	0.245	0.418	+0.173
	CAI-SFT	0.383	0.479	+0.097	0.180	0.307	+0.127
	CAI-DPO (Base)	0.380	0.475	+0.095	0.176	0.311	+0.134
	CAI-DPO (SFT)	0.384	0.482	+0.098	0.182	0.310	+0.127
Gemma	ZS	0.299	0.280	-0.019	0.240	0.237	-0.003
	4-shot	0.284	0.274	-0.010	0.236	0.235	-0.001
	SFT	0.329	0.282	-0.046	0.260	0.235	-0.026
	DPO	0.307	0.287	-0.021	0.247	0.242	-0.005
	CAI-SFT	0.375	0.402	+0.027	0.174	0.211	+0.037
	CAI-DPO (Base)	0.438	0.523	+0.085	0.263	0.352	+0.089
	CAI-DPO (SFT)	0.374	0.402	+0.028	0.173	0.210	+0.037

Table 8: Raw versus byte-length-normalized scoring. Δ columns report normalized minus raw performance.

```

{{USER_REQUEST}}

Available Tools:
{{TOOLS}}

Response A:
{{RESPONSE_A}}

Response B:
{{RESPONSE_B}}

Prefer the response that better follows the
↪ constitution.
Break ties by preferring the shorter, clearer, and
↪ more direct correct response.

Output exactly:
Winner: A
Reason: <one or two short sentences>

or

Winner: B
Reason: <one or two short sentences>

```

D Dataset Construction Details

We built a balanced 9,000-example source set from two When2Call training configurations: the supervised fine-tuning split and the preference-tuning split. The final balanced source contained 3,000 examples for each rough behavior class: `tool_call`, `request_for_info`, and `cannot_answer`.

The class pools were constructed as follows:

- **Tool-call pool.** We used chosen assistant responses from the preference split, retaining only rows whose chosen response was heuristically classified as a tool call.
- **Request-for-information pool.** We used supervised assistant responses that were heuristically classified as follow-up questions requesting missing required information.
- **Cannot-answer pool.** We used supervised assistant responses that were heuristically classified as cannot-answer responses.

The heuristic labeler assigned:

- `tool_call` if the response contained a `<TOOLCALL>` wrapper or began with a JSON object/list corresponding to a tool call,
- `cannot_answer` if it matched common apology/refusal patterns such as `sorry`, `apologies`, `cannot`, `can't`, `or unable`,
- `request_for_info` if it was not already classified as a tool call or cannot-answer response and either ended with a question mark or matched clarification patterns such as `could you`, `please specify`, `please provide`, `or to assist you better`.

After sampling with seed 42, the balanced source file contained:

- 3,000 `tool_call`
- 3,000 `request_for_info`
- 3,000 `cannot_answer`

We then split this balanced source set evenly into a CAI-SFT source split and a CAI-DPO source split. The split was performed class-wise, producing two 4,500-example source sets, each with 1,500 examples per class. No additional filtering or rebalancing was applied after this direct split.

E Probe Training Details

We trained linear probes on a 9,000-example balanced probe-training corpus formed by concatenating the CAI-SFT source split and the CAI-DPO source split. The probe labels were the three rough behavior classes `tool_call`, `request_for_info`, and `cannot_answer`. Probe evaluation used the When2Call test split, restricted to examples whose gold label was in this same three-class label space.

For each example, we rendered the prompt using the same prompting template as the corresponding zero-shot or four-shot baseline. Hidden states were extracted from the *prompt-only* input, without appending the gold answer choices. The representation used for probing was the hidden state of the last non-padding token in the rendered prompt. When truncation was required, prompts were left-truncated so that the end of the prompt, including the current question and answer slot, was preserved.

For each model, we probed three transformer depths:

- the middle layer, defined as $\lceil 0.50L \rceil$,
- a late layer, defined as $\lceil 0.75L \rceil$,
- the final layer, defined as L ,

where L is the number of transformer layers in the model. This yielded layers L14, L21, and L28 for Llama 3.2 3B, and L17, L26, and L34 for Gemma 3 4B.

For each probed layer, we trained a multinomial logistic-regression classifier implemented as a scikit-learn pipeline:

```

StandardScaler()
→
LogisticRegression(
    solver=lbfgs, max_iter=4000)

```

We performed a stratified 80/20 train/dev split over the 9,000 training examples and selected the regularization strength C from the grid $\{0.01, 0.1, 1.0, 10.0, 100.0\}$ based on development-set macro-F1. After selecting the best C , we retrained the classifier on the full 9,000-example training set for that layer.

Probe outputs were compared to the saved behavioral evaluation samples by joining on example UUID. We report probe-vs-gold accuracy and macro-F1, confusion matrices, class-wise precision/recall/F1, and agreement between probe predictions and the model's normalized and raw MCQ predictions.

F Training and Hyperparameter Details

F.1 Model and adapter setup

We used two base instruction-tuned models:

- meta-llama/Llama-3.2-3B-Instruct
- google/gemma-3-4b-it

All SFT and DPO runs used LoRA adapters trained on top of a 4-bit base model (?) The 4-bit quantization configuration used NF4 quantization, double quantization, and a compute dtype matching the run dtype, typically bfloat16. The LoRA configuration was:

- rank $r = 16$
- $\alpha = 32$
- dropout = 0.05
- target modules: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
- bias: none
- task type: causal language modeling

All training used `device_map="auto"` and typically bfloat16 precision.

For the main CAI self-critic condition, the same base model family was used for initial generation, critique, revision, and preference judgment.

F.2 SFT hyperparameters

The SFT hyperparameters were identical for baseline SFT and CAI-SFT:

- validation split: 0.02
- random seed: 42
- maximum sequence length: 2048
- number of epochs: 1.0
- learning rate: 2×10^{-4}
- weight decay: 0.0
- warmup steps: 50
- per-device train batch size: 2
- per-device eval batch size: 2
- gradient accumulation steps: 8
- max steps: -1 (full epoch)
- logging steps: 10
- eval steps: 100
- save steps: 100
- save total limit: 2
- report-to: none
- gradient checkpointing: enabled
- load best model at end: disabled

Baseline SFT trained on the balanced supervised source corpus. CAI-SFT trained on the exported constitution-guided revision dataset.

F.3 DPO hyperparameters

The DPO hyperparameters were identical for baseline DPO, CAI-DPO-from-base, and CAI-DPO-from-SFT:

- validation split: 0.02
- random seed: 42
- maximum sequence length: 2048
- maximum prompt length: 1536
- number of epochs: 1.0
- learning rate: 5×10^{-6}
- weight decay: 0.0
- warmup steps: 10
- $\beta = 0.1$
- per-device train batch size: 2
- per-device eval batch size: 2
- gradient accumulation steps: 8
- max steps: -1 (full epoch)
- logging steps: 10
- eval steps: 100
- save steps: 100
- save total limit: 2

- report-to: none
- gradient checkpointing: enabled

Baseline DPO trained on the original When2Call preference split. CAI-DPO-from-base trained on constitution-guided preference pairs sampled from the base model. CAI-DPO-from-SFT trained on constitution-guided preference pairs sampled from the CAI-SFT policy, starting from the CAI-SFT adapter checkpoint.

F.4 Generation and filtering settings

The CAI generation stages used the following settings in the full cluster wrappers reported in the paper:

- base-generation batch size: 2
- critic/judge batch size: 2
- base-generation prompt cap: 1792 tokens
- critic/judge prompt cap: 1792 tokens

Initial responses. Initial CAI responses were sampled with:

- max_new_tokens = 256
- temperature = 0.7
- top_p = 0.95
- sampling enabled

Each output was parsed and canonicalized. Invalid or malformed outputs were not discarded; they were retained and labeled with structural and validity metadata for downstream critique.

Critiques. Critiques were generated greedily with `do_sample=False`. The full pipeline used `max_new_tokens = 256` for critiques. Critique parsing required the structured fields `Verdict`, `Primary Issue`, and `Critique`. If parsing failed, the row was not dropped; instead, the raw critique text was preserved and used as fallback critique text for the revision stage.

Revisions. Revisions were generated greedily with `max_new_tokens = 256`. If the parsed critique was valid and had verdict `NO_ISSUES`, the pipeline skipped generation and copied the original response exactly. Otherwise, the model generated a revised response, which was canonicalized and structurally validated. The exporter kept non-empty revisions regardless of whether they matched the intended class.

DPO candidate sampling. For each DPO source example, the pipeline sampled up to six candidate attempts:

- max_new_tokens = 256
- temperature = 0.9
- top_p = 0.95
- sampling enabled
- candidate-attempt limit: 6

Sampling stopped early once two distinct canonical responses had been found. Duplicate canonical responses were ignored while searching for a second distinct candidate.

Preference judgment and export filtering.

Preference judgments were generated greedily. The full pipeline used `max_new_tokens = 128` for this stage. Empty candidate pairs were marked invalid with `failure_reason=empty_candidate`, and identical candidate pairs were marked invalid with `failure_reason=duplicate_candidates`. If the judge output could not be parsed, the code performed one retry; if parsing still failed, the record was marked invalid with `failure_reason=preference_parse_failed`. Only valid judged rows were exported to the final CAI-DPO training dataset.